

Milestone 4

Scientific Communication: Paper and Talk

I. GENERAL INFORMATION

Student:	Enea Robaj
Topic:	Dynamic Priority Exchange Server
Seed reference:	Hard Real-Time Computing Systems - Predictable Scheduling Algorithms and Applications Giorgio Buttazzo

II. PROBLEM STATEMENT AND MOTIVATION

A real-time system is any processing system that performs specific functions within a predictable and specific deadline. One of the main goals of these systems is to guarantee predictable timing behavior, even when handling aperiodic tasks. Traditional scheduling approaches may lead to missed deadlines, poor responsiveness, or inefficient CPU utilization under dynamic workloads.

Thus, the *Dynamic Priority Exchange Server* scheduling approach will be investigated in order to understand how effective it is in the handling of aperiodic tasks while preserving timing guarantees for periodic real-time tasks.

The book "Hard Real-Time Computing Systems" by Giorgio Buttazzo will be the main source of this seminar paper, as it offers the most crucial information for this scheduling approach, while other sources are limited.

III. CORE TECHNICAL IDEA

A. EDF Principles

In order to better understand the Dynamic Priority Exchange Server (DPES), it is also important to understand the Earliest Deadline First (EDF) scheduling algorithm, since DPES operates using EDF scheduling principles. The definition of EDF is the following:

Given a set of n -independent tasks with arbitrary arrival times, any algorithm that at any instant executes the task with the earliest absolute deadline among all the ready tasks is optimal with respect to minimizing the maximum lateness[1].

So whenever a new task arrives, sort the ready queue so that the task closest to the end of its period assigned the highest priority.

B. The Dynamic Priority Exchange Server

The Dynamic Priority Exchange (DPE) server is an aperiodic service technique proposed by Spuri and Buttazzo[1]. It is a fixed-priority service algorithm, whose objective is to reduce the average response time of aperiodic requests without compromising the schedulability of hard periodic tasks. DPES, similarly to other dynamic scheduling algorithms, is characterized by high schedulability bounds, which allow the processor to be better utilized, increase the size of aperiodic servers, and enhance aperiodic responsiveness.

IV. TECHNICAL COMPONENTS

The main idea of the algorithm is to let the server trade its runtime with the runtime of lower priority periodic tasks (under EDF this means a longer deadline) in case there are no aperiodic requests pending[1]. In this way, the server runtime is exchanged between periodic tasks, but never wasted. It is simply preserved, even if at a lower priority, and it can be later reclaimed when aperiodic requests enter the system. The Dynamic Priority Exchange (DPE) Server is defined by two main parameters[1]:

- A server period P
- A server capacity C

At the beginning of each server period, the server receives an aperiodic execution capacity equal to C . The deadline of this capacity is set to the deadline d of the current server period [2].

For every instance of a periodic task, an associated aperiodic capacity is maintained. Initially, these capacities are set to 0.

Whenever an aperiodic capacity becomes greater than 0, it is scheduled according to the Earliest Deadline First (EDF) algorithm. In case of equal deadlines, aperiodic capacities are given priority over periodic task instances. Whenever the highest-priority entity in the system is an aperiodic capacity of C units of execution time, a specific exchange mechanism is performed:

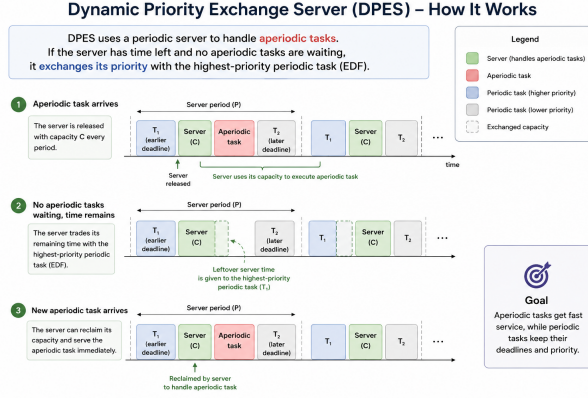


Fig. 1. How priority is exchanged in DPES [3].

- if there are aperiodic requests in the system, these are served until they complete or the capacity is exhausted.
- if there are no aperiodic requests pending, the periodic task having the shortest deadline is executed. A capacity equal to the length of the execution is added to the aperiodic capacity of the task deadline and is subtracted from C .
- If neither periodic nor aperiodic tasks are pending, there is an idle time, and the capacity C is consumed until, at most, it is exhausted.

V. SCIENTIFIC CONTEXT

In real-time systems, periodic tasks are typically associated with strict timing requirements and predictable deadlines. On the other hand, aperiodic task occur unpredictably and often require fast response times.

Several scheduling approaches have been proposed to improve the handling of aperiodic requests while maintaining the timing guarantees of periodic tasks. In the following section, three other scheduling approaches can be found. Although each approach addresses a different aspect of real-time scheduling, they provide useful points of comparison for understanding the position of DPES within the broader field of scheduling.

VI. RELATED APPROACHES

1) *Dynamic Sporadic Server*: The Dynamic Sporadic Server is an EDF-based scheduling mechanism that allocates execution capacity to aperiodic requests while preserving schedulability guarantees. Similar to DPES, it attempts to improve responsiveness for aperiodic workloads. However, DSS relies on replenishment mechanisms that introduce additional runtime management overhead.

2) *Total Bandwidth Server*: The Total Bandwidth Server integrates aperiodic requests into EDF scheduling by assigning deadlines according to available processor bandwidth. This allows aperiodic tasks to be handled without violating processor utilization limits. Compared to DPES, TBS is generally simpler conceptually, although workload characteristics can significantly influence response times.

3) *Stack Resource Policy*: The Stack Resource Policy addresses a different problem. Rather than improving aperiodic responsiveness, SRP focuses on resource sharing and synchronization between tasks. Its primary objective is to prevent priority inversion and ensure safe access to shared resources. Although SRP is not a server mechanism, it provides insights to a approach which is completely different from the server mechanisms which this paper is mostly focused on.

A. Tradeoff Analysis

TABLE I
COMPARISON OF RELATED SCHEDULING APPROACHES [4]

Approach	Primary Goal	Advantages	Limitations
Dynamic Priority Exchange Server	Improve aperiodic responsiveness	Maintains periodic task guarantees and integrates well with EDF	More complex scheduling behavior
Dynamic Sporadic Server	Service aperiodic requests	Good responsiveness and schedulability preservation	Requires replenishment management
Total Bandwidth Server	Bandwidth-based scheduling of aperiodic tasks	Simple EDF integration and efficient bandwidth utilization	Response times depend on workload characteristics
Stack Resource Policy	Resource sharing and synchronization	Prevents priority inversion and improves predictability	Does not directly address aperiodic responsiveness

B. Engineering Consequences

Each approach introduces different engineering trade-offs. Server-based approaches such as DPES, DSS, and TBS primarily focus on balancing responsiveness and schedulability. Faster response times for aperiodic requests generally require additional scheduling complexity and runtime management.

In contrast, SRP focuses on resource management rather than responsiveness. Its benefits become particularly important in embedded systems where multiple tasks compete for shared resources.

From an implementation perspective, DPES introduces additional scheduling logic compared to traditional EDF scheduling. However, this complexity may be justified in systems where unpredictable events must be handled quickly while maintaining hard real-time guarantees.

C. Discussion

The comparison demonstrates that DPES is not an isolated scheduling mechanism, but part of a larger family of EDF-based approaches designed to improve the handling of aperiodic workloads. While DSS and TBS pursue similar objectives while using different methods, SRP addresses primarily resource-sharing challenges.

The choice of scheduling approach depends on system requirements. Some systems may prioritize simplicity, while others may require stronger guarantees regarding responsiveness, resource management, or schedulability.

VII. CRITICAL EVALUATION

In the following section the *Dynamic Priority Exchange Server* will be assessed in order to identify its strengths, weaknesses, assumptions, risks and applicability.

The focus is not to ultimately identify if this approach is the most optimal one, but rather for which assumptions it is useful, where it may fail, and how it transfers to realistic embedded systems contexts.

A. Strengths and Limitations

As the other scheduling approaches, DPES comes with its advantages and limitations. These can be seen below.

Strengths

- Improves responsiveness of aperiodic tasks.
- Preserves EDF schedulability guarantees.
- Overall better processor utilization than simpler approaches.

Limitations

- Increased scheduler complexity
- Additional runtime overhead
- Assumes utilization under EDF Scheduling
- Assumes task parameters are known

If server parameters are incorrect, this may lead to under-utilization of available processor capacity or poor responsiveness

B. Assumptions and Limitations

Assumptions

- Tasks are executed on a single processor system
- Periodic tasks are hard real-time tasks
- Periodic tasks are schedulable under EDF
- Deadlines are predictable and deterministic
- The scheduler can dynamically update priorities during runtime

C. Runtime, Memory and Scalability Implications

Runtime

- Additional scheduling overhead compared to standard EDF.
- More processing power is required to manage server capacities and priorities.

Memory

- Requires extra memory to store server-related data.
- Memory usage increases with the number of tasks.

Scalability

- Should work well for small and medium-sized real-time systems in theory.

As the number of periodic and aperiodic tasks increases, the scheduler must maintain additional deadline and capacity information, which could increase runtime overhead and implementation complexity.

D. Formal Implementation

Implementation in UPPAL or C/C++ to be added.

VIII. DPES APPLICATION EXAMPLES

Suitable Applications

- Automotive control systems
- Industrial automation
- Robotics
- Avionics
- Systems with many periodic tasks and occasional asynchronous events.

Unsuitable Applications

- Small microcontrollers with extremely limited resources.
- Systems where simple fixed-priority scheduling is sufficient.
- Applications with almost no aperiodic workload.

DPES is most suitable for systems that must handle unpredictable events while maintaining strict timing guarantees for periodic tasks, and less suitable for simple or highly resource-constrained systems [5].

IX. CONCLUSION

This work has explored the Dynamic Priority Exchange Server, analyzing how it works, how it compares with other approaches and critically evaluating it.

ACKNOWLEDGMENT

AI USAGE DOCUMENTATION

REFERENCES

- [1] G. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011.

- [2] 1232602, "Ai usage protocol for milestone 1: Dynamic priority exchange server," Submitted AI usage protocol notebook and JSON file, 2026, machine-readable documentation of AI-assisted scientific work; course: ELE Real-Time Systems; protocol version: 0.2; documented reasoning episodes: 1; JSON file: ai_usage_protocol_1232602_Milestone_1.json.
- [3] —, "Ai usage protocol for milestone 4: Dynamic priority exchange server," Submitted AI usage protocol notebook and JSON file, 2026, machine-readable documentation of AI-assisted scientific work; course: ELE Real-Time Systems; protocol version: 0.2; documented reasoning episodes: 1; JSON file: ai_usage_protocol_1232602_Milestone_4.json.
- [4] —, "Ai usage protocol for milestone 2: Dynamic priority exchange server," Submitted AI usage protocol notebook and JSON file, 2026, machine-readable documentation of AI-assisted scientific work; course: ELE Real-Time Systems; protocol version: 0.2; documented reasoning episodes: 2; JSON file: ai_usage_protocol_1232602_Milestone_2.json.
- [5] —, "Ai usage protocol for milestone 3: Dynamic priority exchange server," Submitted AI usage protocol notebook and JSON file, 2026, machine-readable documentation of AI-assisted scientific work; course: ELE Real-Time Systems; protocol version: 0.2; documented reasoning episodes: 1; JSON file: ai_usage_protocol_1232602_Milestone_3.json.